antiHeuristik

PATHEI MATHOS ARA PRŌTAI ARCHAI

cargo cult engineering × return to monke

auto-generated 2026-03-29
source: grind/theo/ index hierarchy

Table of Contents

1. Backend Development

backend development patterns, APIs, system design

1.1. patterns

software engineering patterns: abstraction, retry, event sourcing, API integration, and observability

Proxy Abstraction Layer

LiteLLM, database ORMs, filesystem VFS layers — same pattern. Wrap a provider-specific interface behind a unified API so callers don't couple to transport details. The trade-off is always portability vs provider-specific optimizations. For LLM clients: you gain model swappability (LMStudio → Ollama → cloud) but lose access to provider-specific features (LMStudio's speculative decoding config, Ollama's keep_alive, etc.).

The discipline: keep the abstraction thin. A client handles connection, retry, and serialization. Business logic (prompt construction, memory management, eval) stays in the caller. The moment you add prompt templates or conversation history to your client, you've created a god object.

Repository Pattern Applied to LLM Calls

The async client is essentially the Repository pattern — abstract "how do I talk to the model" so consumers think only in terms of "send messages, get response." This is the same reason you don't scatter SQL queries across your codebase. One place to change connection logic, retry policy, or provider.

Exponential Backoff

Standard retry pattern: wait 2^attempt seconds between retries. Critical for local inference where the model might be mid-generation on another request and temporarily unresponsive. Without backoff, rapid retries just pile up and make things worse (thundering herd on your own machine). Three retries with 1s/2s/4s waits covers transient failures without making the user wait too long on genuine failures.

Event Sourcing for Conversation State

The conversation runner implements an append-only event log where each event is a `(user_turn, assistant_response)` pair. This is the same pattern used in CQRS/event-sourced systems — the log *is* the state, and you can derive any view (eval scores, latency charts, token counts) by replaying it.

Key properties:

- **Deterministic inputs**: scripted user turns are fixed; only model responses vary between runs
- **Incremental persistence**: save after each turn, not at the end. A crashed run still produces usable partial data — you lose one turn, not the whole conversation
- **Separation of capture and judgment**: the runner writes raw data, the eval layer scores it. This means you can re-score the same run with different eval criteria without re-running the (expensive) inference

This is also why the runner uses `complete()` (non-streaming) rather than `stream()` — for benchmarking, you want the full response as a single atomic artifact. Streaming is for UX, not for eval.

API Documentation vs API Reality (Impedance Mismatch)

When different endpoints in the same API name the same concept differently, consumers always hit this wall. LMStudio's listing endpoint returns `loaded_instances[].id`, but the unload endpoint expects `{"instance_id": ...}` — same value, different key name. Similarly, models loaded via UI get short identifiers

(`qwen3.5-9b`) while the API uses full paths (`qwen/qwen3.5-9b`).

This is the **impedance mismatch** problem — a term borrowed from electrical engineering (and famously applied to ORMs by Ted Neward). Two representations of the same entity don't map cleanly. The defensive patterns:

1. **Dump raw responses during integration**. Never trust docs alone — the first thing you build against a new API is a diagnostic that prints the actual response shape. You debug from reality, not from specs.
2. **Bidirectional matching**. When comparing identifiers across system boundaries, check both directions: `a in b or b in a`. One system may use a prefix, namespace, or abbreviation the other doesn't. Unidirectional substring matching is a latent bug.
3. **Canonical identifiers**. If you control the code but not the API, normalize identifiers early (strip prefixes, lowercase) and compare canonical forms. Don't let different naming conventions leak into business logic.

The broader lesson: integration bugs are almost never in the logic — they're in the assumptions about data shape. Treat every external API response as untrusted structure until you've verified it empirically.

Observability Over Configuration

A system is **observable** (control theory, Kalman 1960) when its internal state can be determined from its outputs alone. Applied to software: prefer reading actual state over trusting declared constants.

Concrete example: GPU VRAM. A hardcoded `GPU_VRAM_GB=12` in `.env` is a lie the moment you run on a different machine, a driver update changes reported memory, or another process allocates VRAM. Reading `nvidia-smi` at runtime gives ground truth — self-correcting, no manual tuning, works on any machine.

The general principle from **twelve-factor app** methodology (Wiggins, 2011): configuration is for things that *vary between deploys* and *cannot be detected* (API keys, endpoint URLs). Hardware state is not configuration — it's observable system state. Treating observable state as configuration creates a second source of truth that inevitably drifts from the first.

When to prefer detection over config:

- **Hardware**: VRAM, CPU cores, disk space — always detectable, always drifting
- **Runtime state**: loaded models, running processes, network reachability — poll, don't assume
- **Versions**: library versions, API versions — query, don't hardcode

When config is still correct:

- **Secrets**: API keys, tokens — not detectable
- **Policy**: base URLs, retry limits, timeouts — human decisions, not physical state
- **Overrides**: when you *want* to lie to the system (test with 4GB VRAM limit on a 12GB card)

Thinking Model Output Routing

Not all models put their response in the same field. Standard instruction-tuned models return content in `choices[0].message.content`. But thinking/reasoning models (Qwen 3.5, DeepSeek-R1, etc.) may split their output:

- `content`: the final user-facing response (may be empty if all tokens went to thinking)
- `reasoning_content`: the internal chain-of-thought (the "thinking" block)

Qwen 3.5 at low `max_tokens` (e.g. 256) spends its entire token budget on `reasoning_content` and returns `content: ""`. At higher budgets (2048+), it finishes thinking and writes the actual response to `content`. LMStudio separates these fields at the API level — the same model served through a different runtime might concatenate them or use `<think>` tags instead.

This is a specific instance of the **polymorphic response** problem: the same API endpoint returns structurally different responses depending on the model loaded behind it. The defensive pattern is a **fallback chain** — check `content` first, fall back to `reasoning_content`, and fail explicitly if both are empty. Never assume the response shape is stable across models.

The broader lesson connects to Postel's Law (the Robustness Principle): "Be conservative in what you send, be liberal in what you accept." When consuming API responses, handle all documented fields even if only one model uses them. When producing requests, use the most standard format possible.

2. Deep Learning

deep learning fundamentals, architectures, and training

2.1. attention

attention mechanisms, architecture comparison, scaling behavior, quantization, and precision tradeoffs

Mechanisms & Architectures

Grouped Query Attention (GQA)

Shares K and V projections across groups of query heads. If a model has 32 query heads but only 8 KV heads, every 4 query heads share the same keys. This cuts KV cache size by 4x but trades representational capacity — the shared keys are a compromise optimized for no single head. At long context, this compromise hurts: when two heads in the same group need to attend to different distant facts, the shared keys can't serve both well.

Qwen3.5 uses GQA. This means it can *fit* longer contexts in VRAM, but recall quality at those lengths is slightly worse than equivalent MHA (multi-head attention) would give.

Flash Attention

Tiles the attention computation to avoid materializing the full N×N attention matrix in GPU HBM (slow main memory). Instead, streams blocks of Q, K, V through fast on-chip SRAM. Result is bit-for-bit identical to standard attention but:

- Memory: O(N) instead of $O(N^2)$
- Speed: 2-4x faster at long contexts (eliminates HBM round-trips)

Flash Attention is a pure engineering win — it doesn't change what's computed, just how the hardware executes it. It does NOT fix attention precision issues.

Key concept: the **Roofline model** — attention is memory-bound, not compute-bound. The GPU can do the math 10x faster than it can fetch the data. Flash Attention closes that gap.

KV Cache Mechanics

Stores K and V vectors for every past token so they don't need recomputation. At 20K tokens with 32 layers:

- KV cache can reach ~10GB for a 9B model at FP16
- This competes with model weights for VRAM
- Each new token computes attention against the *entire* accumulated cache

With GQA + Q4 quantization, the K vectors in cache are derived from quantized weights (already slightly wrong). Errors accumulate as cache grows — they don't cancel out because quantization systematically reduces the dynamic range of attention scores (peaks get lower, valleys get higher).

Static Pre-Allocation vs Dynamic KV Cache Growth

LMStudio pre-allocates the full KV cache at model load time based on the `context_length` parameter. A 9B Q4 model loaded with 262K context consumed ~11.4GB of 11.94GB VRAM — the model weights are 6.55GB, and the remaining ~5GB was pre-allocated KV cache for 262K tokens. Reducing to 64K dropped usage significantly.

This is the **static allocation** pattern: reserve the worst-case memory budget upfront, guarantee no mid-run failures. The alternative is **dynamic allocation** where KV cache grows per-token — smaller footprint on short conversations but risks OOM at turn 45 of 48 (losing the entire run).

The trade-off maps directly to real-time systems engineering (avionics, medical devices) where `malloc` at runtime is banned. The reasoning is identical: if you allocate dynamically, you must prove the system can't exceed its budget under any input — which is equivalent to computing the worst case anyway. Static allocation makes the worst case the *only* case.

For benchmarking, static is strictly better: a crash partway through a run wastes more than the extra VRAM would. For production chat (variable-length conversations), dynamic allocation with a hard cap and graceful degradation is more practical — vLLM's PagedAttention takes this approach.

Practical rule: set `context_length` to the actual maximum you'll need, not the model's advertised maximum.

PagedAttention

Used in vLLM. Extends Flash Attention's idea to the KV cache itself — applies virtual memory paging concepts to GPU memory for dynamic context lengths. Relevant for production serving but not for single-session testing.

Architecture Comparison

Attention Architecture

Three different attention patterns, three different predictions:

Qwen — GQA (full global) Every layer, every token attends to every other token. Shared K/V heads reduce memory but every layer has global reach. Degradation should be gradual and uniform across fact positions — the softmax dilution curve.

Gemma — Sliding Window 5:1 5 local layers (1024 token window) per 1 global layer. Creates a **recency bias at the architecture level**:

- Recent facts: visible to all 6 layers (local + global)
- Distant facts: visible to only 1/6 layers (global only)

Prediction: steeper recall drop-off for distant facts, flatter for recent. The curve shape should be visibly different from Qwen's.

Phi-4 — Full Attention Full attention at every layer, like Qwen's GQA but without the shared K/V heads. At 3.8B params, each layer is smaller than Qwen's — less capacity per attention operation but no architectural bottleneck to distant tokens.

If all three architectures produce the same degradation curve → attention architecture doesn't matter, it's just about total capacity and precision. If they diverge → architecture is a load-bearing variable. That's the finding.

Capacity vs Precision

The central trade-off for local inference: more parameters at lower precision, or fewer parameters at higher precision, within the same VRAM budget?

	Qwen 3.5 9B Q4	Gemma 3 4B Q8	Phi-4 3.8B Q8
Params	9B	4B	3.8B
Precision	~1-1.5 digits	~3 digits	~3 digits
VRAM	6.6GB	4.98GB	4.08GB

This is the local-inference version of the bias-variance trade-off:

- **More params** = more capacity to route attention precisely (lower bias), but Q4 adds noise to every weight (higher variance)
- **Fewer params** = less routing capacity (higher bias), but Q8 keeps routes cleaner (lower variance)

At short context, capacity dominates — not enough tokens for quantization noise to accumulate. At long context, noise compounds turn-over-turn in the KV cache, and precision might overtake capacity. The crossover point — if it exists — is what this eval measures.

How to interpret:

- If Qwen 9B Q4 degrades faster than Gemma 4B Q8 on distant probes → **precision > capacity** for long-context recall
- If Qwen holds better → **capacity > precision**, scaling beats quantization quality
- If they degrade at the same rate but on different fact types → the dimensions are orthogonal

The Lineup

Qwen 3.5 9B — Q4_K_M (~6.6GB)

Role: High-capacity, low-precision baseline.

- 9B params, GQA (grouped query attention), 128K native context
- Full attention at every layer — every token attends to every other token
- Q4_K_M quantization reduces weights to ~1-1.5 digits of effective precision
- Most parameters in the lineup = most representational capacity for routing attention
- Q4 = lowest precision = most vulnerable to attention score flattening at long context

Prediction: Should hold recall longest due to raw capacity, but Q4 noise will compound turn-over-turn in the KV cache. Expect gradual degradation starting around turn 30-40, with the lost-in-the-middle zone (facts at 40-60% depth) failing first.

Gemma 3 4B — Q8_0 (4.98GB)

Role: Sliding window architecture + high precision.

- 4B params, sliding window (1024) interleaved with global attention at 5:1 ratio, 128K native
- Only 1/6 of layers do global attention — the other 5/6 only see the nearest 1024 tokens
- Q8_0 = ~3x the effective precision of Q4
- RoPE base frequency 1M (vs 10K in Gemma 2)
- KV cache overhead <15% vs ~60% for global-only architectures

Prediction: Recent facts should be recalled extremely well (all 6 layers see them). Distant facts degrade faster than Qwen because only global layers (1/6) carry that information. The degradation curve should have a different *shape* — steeper drop-off for distant facts, flatter for recent. If we observe this, it's direct evidence that attention architecture shapes the recall curve, not just context length.

Phi-4 Mini Instruct — Q8_0 (4.08GB)

Role: Microsoft architecture baseline, control for reasoning ablation.

- 3.8B params, full attention at every layer, 128K native context
- Phi-4 architecture — distinct from both Qwen's GQA and Gemma's sliding window
- Trained on synthetic, high-quality data (Microsoft's approach)
- General-purpose instruction tuning
- Smallest model in the lineup

Prediction: Full attention gives it the best theoretical reach to distant facts per-layer, but 3.8B params means less capacity to maintain distinct fact representations. Q8 precision helps. Expect a capacity-limited failure pattern — may start confusing facts with each other (merging details) rather than losing them entirely.

Phi-4 Mini Reasoning — Q8_0 (4.08GB)

Role: Reasoning-tuned ablation of Phi-4 Mini Instruct.

- Identical architecture, identical quant, identical size
- Only difference: fine-tuned on synthetic reasoning-dense data with chain-of-thought
- CoT reasoning generates extra tokens per response that consume context budget

Prediction: Two competing effects:

1. **CoT helps recall** — reasoning forces the model to explicitly re-derive facts from context rather than pattern-matching from memory. This could improve grounding and reduce parametric override.
2. **CoT hurts context budget** — extra reasoning tokens (200-500 per response) consume ~10-25K tokens across 50 turns. Despite 128K window, effective available context shrinks.

3. **Domain mismatch** — math-reasoning training may have narrowed attention patterns for structured problems, weakening free-form conversational recall.

The delta between Phi-4 reasoning and Phi-4 instruct isolates the effect of reasoning-specific training on conversational context grounding.

Scaling & Degradation

Lost-in-the-Middle Effect

Liu et al., 2023. LLMs recall facts placed at the beginning and end of context far better than facts buried in the middle. In a 100-turn conversation, facts at turns 30-60 sit in the attention dead zone — even if the context window hasn't overflowed. Probe design must control for *position* of the planted fact, not just distance in turns.

Attention Score Degradation at Long Context

Every token gets an attention score — "how relevant is this token to what I'm generating now?" At short context (~500 tokens), relevant tokens score 50-100x above noise. At long context (~20K tokens), softmax distributes across all positions — relevant tokens might only score 40x above noise, and there are far more noise positions competing. The signal-to-noise ratio shrinks with length even without any quantization.

Effective vs Advertised Context

Full-precision model might have perplexity ~8 at 4K tokens and ~12 at 32K. Q4_KM might be ~8.5 at 4K but ~16+ at 32K. The gap widens with context length. Quantization benchmarks on short prompts are misleading for long-context use cases — always test at the context lengths you actually plan to use.

Expected Degradation Pattern (Q4 9B model)

- Turns 1-20: nearly identical to full precision, recall probes pass
- Turns 20-40: subtle hedging ("I believe you mentioned…"), slightly less precise recalls, occasional fact merging
- Turns 40-60: recall probes for early facts start failing, confabulation appears (plausible but wrong)
- Turns 60+: coherence collapse — contradictions within 5 turns, repetition, thread loss

Exact turn numbers shift based on token density per turn. High-density conversations hit the wall 2-3x sooner than low-density.

Compound Effect of Optimizations

GQA (trades precision) + Flash Attention (free) + Q4 quantization (trades precision) = quality degrades faster than any single technique would suggest. The “effective context window” under all three may be ~40-50% of the advertised maximum. Nobody benchmarks the compound effect — they benchmark each optimization in isolation and assume linear composition. They don’t compose linearly.

Predicted Degradation Patterns

Based on architecture and quantization theory:

Qwen 3.5 9B Q4

```
Turn 1–20: near-perfect recall, Q4 noise negligible Turn 20–40: subtle hedging on independ
```

Shape: gradual linear degradation, worst in the 40-60% depth zone.

Gemma 3 4B Q8

```
Turn 1–20: strong recall, Q8 precision keeps scores sharp Turn 20–40: distant facts start
```

Shape: bifurcated — steep for distant, flat for recent. Different curve shape from Qwen.

Phi-4 Mini Instruct Q8

```
Turn 1–20: good recall, full attention + Q8 helps Turn 20–40: capacity limits surface – fa
```

Shape: uniform degradation, earlier onset than Qwen but without Gemma’s bifurcation.

Phi-4 Mini Reasoning Q8

```
Turn 1–20: possibly better recall than instruct (CoT re-derives facts) Turn 20–40: CoT con
```

Shape: depends on which hypothesis holds — could mirror instruct (shifted left by CoT overhead) or outperform it (reasoning improves per-token recall).

Quantization

Perplexity

How “surprised” a model is by the next token. Lower = more confident and accurate. Defined as `perplexity = 2^(cross-entropy)`. When papers report “0.1 increase in loss from quantization,” that sounds small, but exponentiated over thousands of tokens across a long conversation, it means materially more wrong-token choices.

Q4 Quantization and Attention Precision

Weights stored with ~1-1.5 digits of effective precision (vs ~3-4 for FP16). Attention scores are computed from Q, K matrices derived from these weights. At short context, the gap between relevant and irrelevant attention scores is huge — small rounding doesn’t flip rankings. At long context, scores are already small and close together. Q4 rounding introduces noise on the *same scale as the signal difference*. The model literally cannot distinguish between the token it should attend to and 50 nearby irrelevant tokens.

Systematic Flattening

Quantization doesn’t add random noise — it *systematically reduces the dynamic range* of attention scores. Peaks get lower, valleys get higher. The model’s ability to say “THIS token matters, those don’t” gets weaker with every turn added to context. This is directional, not stochastic — it consistently degrades retrieval.

3. Natural Language Processing

natural language processing, tokenization, and language models

3.1. evaluation — LLM evaluation: failure modes, scoring frameworks, behavioral patterns, and experimental confounds

- Failure Modes
- Methods & Frameworks
- Scoring Framework
- Behavioral Patterns
- Experimental Confounds

3.1. evaluation

LLM evaluation: failure modes, scoring frameworks, behavioral patterns, and experimental confounds

Failure Modes

When an LLM fails to recall a fact from earlier in a long conversation, what caused the failure? Possible root causes:

1. **Attention dilution** — softmax spread scores across too many tokens, signal drowned in noise
2. **Quantization noise** — reduced weight precision corrupted attention score rankings
3. **Architectural bottleneck** — model's attention pattern physically can't reach the target tokens
4. **Parametric override** — model's training data contradicted context, and training won
5. **Context truncation** — the information was silently dropped from the context window
6. **Capacity limitation** — model has too few parameters to maintain distinct representations of many facts

A single model can't distinguish these. You need a controlled lineup where each model changes one variable.

Methods & Frameworks

Needle in a Haystack (NIAH)

Kamradt, 2023. Standard test: bury a single random sentence in filler text, check retrieval at varying depths. Most models fail retrieval when the needle is in the 40-60% depth range of context. Probe design for long-context evaluation improves on NIAH: (a) multiple needles that relate to each other, (b) haystack is actual conversation not filler, © probes test reasoning over facts, not just verbatim retrieval.

Scripted Dialogue Evaluation

Deriu et al., 2021. Deterministic test scripts are "conversation unit tests." The key advantage over live testing: you can re-run the exact same conversation after changing strategies (stuffing → sliding window → RAG) and get apples-to-apples comparison. Without deterministic scripts, you're benchmarking conversation content, not context strategy.

Probe Design

Probes are injected at fixed intervals (every ~10 turns) testing recall of specific facts. Probe types:

- **Direct recall**: "How many games was Kemp's streak?" — tests verbatim retrieval
- **Cross-topic recall**: after switching topics, circle back — tests whether topic context helps or hurts retrieval
- **Contradiction resistance**: user states incorrect "correction" — tests whether model holds ground or caves (sycophancy)
- **Comprehensive recall**: "list all the facts we discussed" — tests breadth of memory

Ground Truth Control

Using one real fact mixed with fabricated ones. If the model recalls real facts more reliably, training data leakage is inflating results. This distinguishes context recall from parametric knowledge.

Instruction Hierarchy and Fact Placement

Wallace et al., 2024 (OpenAI). Models are trained with a trust priority stack: system prompt > user instructions > retrieved context. Facts seeded in user turns sit in the middle of this hierarchy — moderate trust. Moving facts to the system prompt would likely improve recall but wouldn't reflect real conversation dynamics. For benchmarking context strategies, user-turn placement is the right choice because it matches production conditions (user says something, model should remember it).

Reasoning vs General Tuning

The cleanest comparison: same architecture, same quant, same size — only the fine-tuning differs.

Hypothesis A — reasoning training helps conversational recall: CoT forces the model to explicitly engage with context rather than relying on pattern matching. When the model "thinks through" a probe, it performs active retrieval over context. This is analogous to how human study works — actively recalling beats passive recognition.

Hypothesis B — reasoning training hurts conversational recall: Math-reasoning training narrows attention patterns to favor structured, sequential reasoning. Conversational recall requires broad, associative attention over unstructured dialogue. The model's attention "muscles" are trained for the wrong task.

Hypothesis C — reasoning training is neutral for recall, but CoT consumes context: The training doesn't change recall quality, but the extra tokens generated by reasoning chains eat into the 128K budget, causing earlier context pressure. This would show as identical recall-per-remaining-context but worse recall-per-turn-number.

How to detect which hypothesis holds:

- Normalize recall scores by remaining context tokens (not turn number)
- If reasoning scores the same as instruct when normalized → Hypothesis C
- If reasoning scores better even after normalization → Hypothesis A
- If reasoning scores worse even after normalization → Hypothesis B

Training Pipeline Diversity

Three companies, three philosophies:

Pipeline	Models	Approach
Alibaba (Qwen)	Qwen 3.5 9B	Massive multilingual data, aggressive instruction tuning, known agreeability
Google (Gemma)	Gemma 3 4B	Distilled from Gemini research, multimodal training, conservative outputs
Microsoft (Phi)	Phi-4 Instruct + Reasoning	Synthetic data focus, quality over quantity, reasoning emphasis

If models from different pipelines fail on different facts or in different ways, the training data composition is a variable — not just architecture and precision.

Run-Until-Failure vs Fixed Turn Count

Run each path until the model fails 3 consecutive probes rather than a fixed turn count. This gives the natural breaking point per path instead of an arbitrary cutoff. More informative for establishing the degradation curve.

Scoring Framework

Recall Score

- **Pass**: correct recall of key details
- **Hedge**: partially right or explicitly uncertain ("I believe…")
- **Fail**: wrong answer or omission
- **Confabulate**: confidently wrong — model generates plausible but incorrect details

Hedging is actually a better failure mode than confabulation — a model that knows it doesn't know is more useful than one that makes things up.

Conflict Behavior

Standard pass/hedge/fail/confabulate scores measure *whether* the model recalled correctly. But *how* the model handles parametric-contextual conflict is equally important, especially for RAG applications. Every probe response should also be tagged with a conflict behavior:

- **Deferred**: model used context faithfully, no sign of parametric interference. Good for RAG grounding.
- **Overrode**: model's answer matches real-world data instead of planted context. Parametric memory won. Bad for RAG, but shows guardrail strength.
- **Flagged**: model acknowledged uncertainty or conflict ("you mentioned X, though I recall Y"). Calibrated deference — the gold standard. Rare.
- **Caved**: on contradiction probes only — model abandoned its earlier correct answer and agreed with the user's wrong correction. Sycophancy.

These are orthogonal to pass/fail. A model can *pass* (correct recall) with `deferred` behavior, or *fail* with `overrode` behavior (wrong answer that happens to match real NBA data). Tracking both dimensions reveals whether failures are attention/recall problems or alignment/grounding problems — different root causes requiring different solutions.

Key diagnostic: if a wrong answer matches real-world data, that's `overrode`. If it's a novel wrong answer, that's plain recall failure. The distinction matters for choosing between strategies (longer context won't fix override; better prompting might).

These combine into a 4x4 matrix. Example interpretations:

- Pass + Deferred = ideal RAG behavior
- Fail + Overrode = parametric memory won (training data leakage)
- Hedge + Flagged = model knows it doesn't know (best failure mode)
- Pass + Caved = model changed to user's wrong correction (sycophancy)
- Confabulate + Deferred = model used context but reconstructed incorrectly (schema confabulation)

Behavioral Patterns

Sycophancy at Long Context

Perez et al., 2022 (Anthropic). Models tend to agree with the user even when the user is wrong. This gets *worse* at longer contexts because:

- Model's confidence in its own earlier statements decays (attention scores to its own past outputs weaken)
- Recent user pressure stays strong (recency bias in attention)
- The model "forgets" that it's right and defaults to agreeing with the closest strong signal

Gaslighting probes ("actually you said X" when the model said Y) directly test this. The turn at which the model stops pushing back is a direct measure of effective context for self-consistency.

Schema-Consistent vs Schema-Inconsistent Retrieval

Facts that fit a narrative schema (e.g., the Warriors resurgence arc: trade → coaching change → rookie breakout → playoff push) are easier to recall because the schema acts as a retrieval cue. The model can reconstruct from the narrative even if exact attention is weak.

Independent facts (e.g., "Thunder average age 23.1") have no schema support — they're pure episodic recall from context.

Critical distinction: reconstruction (schema-based) is where **confabulation hides**. The model reconstructs a plausible-sounding answer from the schema but gets specific details wrong. This is why probes need to test exact numbers, not just topic recall.

The stress test should measure schema-consistent and independent facts separately to understand which type of recall degrades first.

Parametric vs Contextual Memory Conflict

Longpre et al., 2021. LLMs have two knowledge sources — parametric (baked into weights during training) and contextual (provided in the prompt). When they conflict, models show a *popularity bias*: they defer to parametric memory for well-known facts and to context for obscure ones.

By anchoring fabricated facts to the 2021-22 NBA season — a period the model *definitely* has training data for — every fact becomes a direct collision with parametric memory. The model's behavior under conflict (cave to training data, hedge, or faithfully use context) is a direct measurement of contextual grounding strength. This is exactly what RAG systems struggle with in production: user-provided context that contradicts what the model "knows."

Contrast with future-dated facts (e.g., 2025-26 season): the model has no competing knowledge, so it *must* use context. That tests recall but not the harder problem — can the model prioritize context over its own priors?

Each fact directly contradicts real-world data (e.g., OKC was actually the youngest team, not oldest; 3PA rate was rising, not falling). This turns the eval from a passive recall test into a **parametric conflict test**:

High-conflict facts (contradict well-known real data):

- Salary cap ($163.5M vs real ~$112M)
- OKC roster age (27.6 oldest vs reality: one of youngest)
- Three-point rate (31.7% declining vs reality: ~37% rising)
- Celtics road record (11-game losing streak vs reality: strong road record)

Low-conflict facts (fictional entities, no real data to compete):

- Marcus Kemp, Terrence Okafor, Viktor Dragas (fictional players)
- Devin Harlow (fictional coach)
- Balboa Park Arena, Arena CDMX (fictional venues)

How to interpret probe failures:

- Model's wrong answer matches real 2021-22 NBA data → **parametric override** (training won over context)
- Model's wrong answer is novel (doesn't match real or planted data) → **recall failure** (attention/capacity issue)
- Model recalls correctly AND flags uncertainty → **calibrated deference** (ideal RAG behavior)
- Model caves to user contradiction of planted facts → **sycophancy** (grounding failure)

Each model's RLHF training will handle this tension differently. Qwen is known to be agreeable (predicts high deference, low override). Phi-4's Microsoft RLHF is less characterized. Gemma's Google training may be more cautious. The four-way comparison reveals how training pipeline shapes the grounding-vs-guardrails balance.

Knowledge Conflict as Eval Dimension

Most NIAH benchmarks use neutral filler text that doesn't conflict with training data. That tests attention/retrieval in isolation. Planting *contradictory* facts (OKC oldest when training says youngest, 3PA declining when training says rising) tests a compound capability: retrieval + override.

This is harder and more production-relevant. Facts should be categorized by expected parametric conflict strength:

- **High conflict**: salary cap number, OKC roster age, 3PA trend, Celtics road record — these contradict well-known real data
- **Low conflict**: fictional player names (Marcus Kemp, Terrence Okafor, Viktor Dragas), fictional venues (Balboa Park Arena, Arena CDMX) — no real data to compete with

When a high-conflict fact fails a probe, it could mean either (a) context recall failed or (b) parametric memory overrode context. Distinguishing the two: if the model's wrong answer matches real-world data, that's override. If it's a novel wrong answer, that's recall failure. Track both separately.

Conversational Grounding

Clark & Brennan, 1991. In human dialogue, facts become "common ground" through acknowledgment — one party states, the other confirms or builds on it. If the model acknowledges a fact in its response ("yeah the Kemp trade really shook things up"), that's stronger grounding than if the fact just sits in a user turn unacknowledged.

This matters for path design: facts the model explicitly engages with in its responses may be recalled better later — the model's own output acts as a second encoding of the fact in context. Facts that appear only in user turns and get a generic response have weaker grounding. The eval should track whether model acknowledgment correlates with later recall success — grounded facts vs ungrounded facts.

Experimental Confounds

Silent Truncation

When context exceeds the window, most inference engines silently truncate from the beginning — exactly where early-seeded facts live. The model doesn't error, it just loses turns 1-N without indication. This makes truncation-caused failures look like recall failures.

Mitigation: All four models have 128K native context. At ~300-400 tokens per turn pair, even HIGH path (48 turns) should stay under 20K tokens for conversation content. Phi-4-reasoning's CoT chains are the risk — monitor token count per turn.

Context Overflow vs Recall Degradation

These are fundamentally different failure modes:

- **Recall degradation**: information is present in context but attention can't find it
- **Context overflow**: information is physically truncated from context

Mixing them in the same analysis contaminates the benchmark. Models with short native context (e.g., 33K) should be excluded from long-context evaluations — they would overflow, making it impossible to distinguish recall failure from truncation.

Domain Specificity

Math-reasoning training may cause domain mismatch on conversational recall. If Phi-4-reasoning underperforms Phi-4-instruct, it could be domain mismatch rather than a fundamental property of reasoning training.

Mitigation: Compare the two variants directly. If reasoning scores worse, check *how* it fails — domain mismatch shows as generic/vague responses ("I'm not sure about sports details"), while attention/recall failure shows as confident wrong answers or fact merging.

YaRN Extension vs Native Context

Models with YaRN-extended context (e.g., Qwen3 8B: 40K native → 128K extended) introduce positional embedding artifacts at extended ranges. Quality degrades at positions beyond the native training length.

Mitigation: Use models with native long context. No YaRN extensions.

CoT Token Overhead

Reasoning models generate chain-of-thought tokens that consume context budget but aren't "conversation content." A reasoning model at turn 40 may have consumed 30K tokens of context, while a non-reasoning model at turn 40 consumed only 15K.

Mitigation: Track cumulative token count per turn. Normalize recall scores by remaining context (tokens), not by turn number, for the reasoning vs instruct comparison.